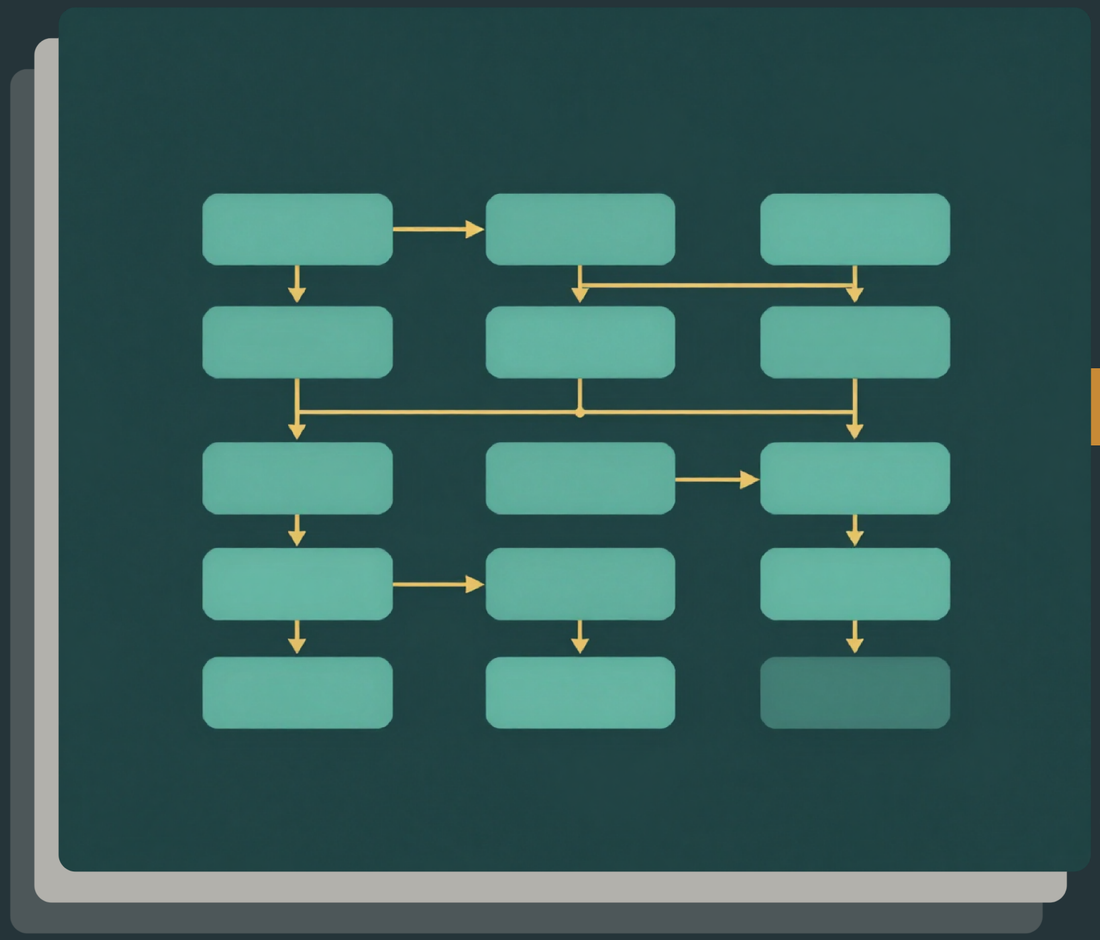
SOFTWARE ARCHITECTURE PATTERNS

PRESENTED BY : MIGUEL DAVID & WILMER RONDEROS



WHAT IS SOFTWARE ARCHITECTURE?

Definition, Structure & Purpose

Software architecture is the high-level structure of a software system. It organizes components, responsibilities, communication, and data flow. Its purpose is to guide development decisions and ensure quality attributes such as performance, scalability, and maintainability.

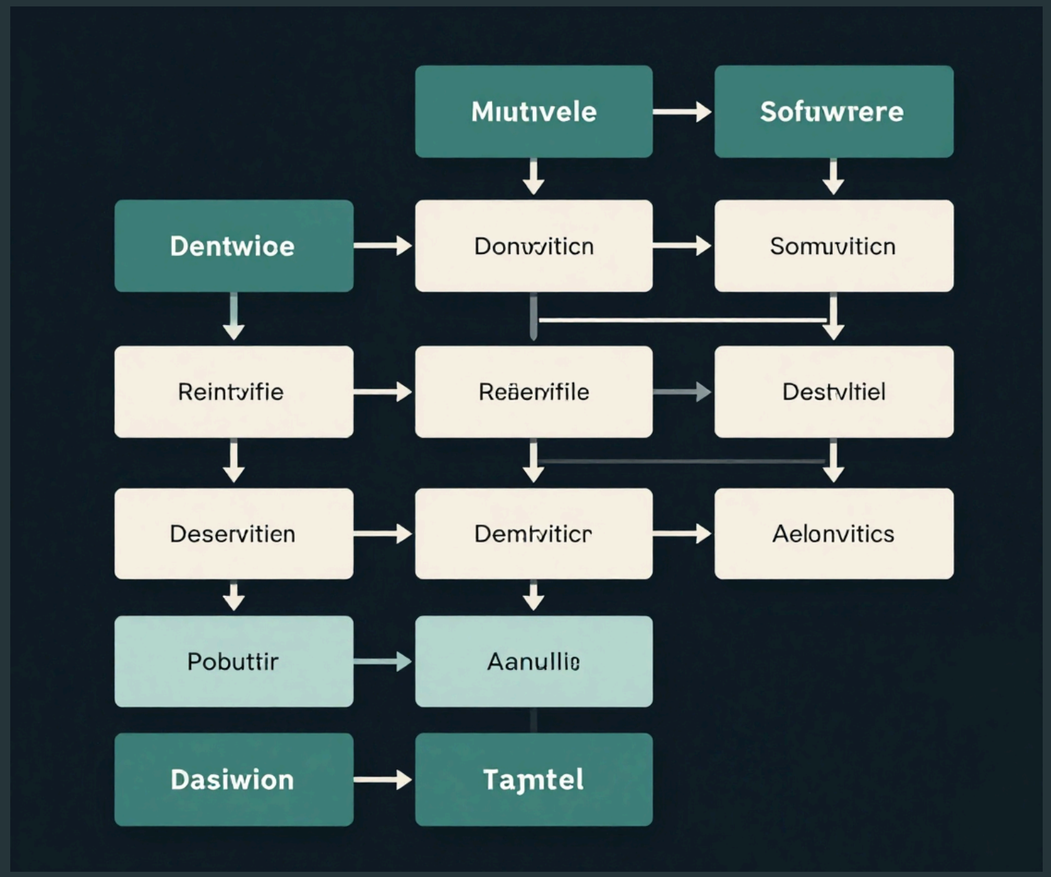
ARCHITECTURAL PATTERN

Reusable Solution
A general, proven approach to organizing a software system's structure and components.

Design Guideline
Not ready-to-use code — it is a conceptual template adapted to specific project needs.

An architectural pattern is a reusable, general solution for recurring structural problems in software design. It defines how components are organized and interact, serving as a high-level guideline — not executable code.

MAIN COMPONENTS



Core Elements of Software Architecture

Components: modular units that encapsulate specific functionality
Connectors: pathways enabling communication between components
Data: information exchanged and stored across the system
Responsibilities: defined roles and duties assigned to each part
Constraints: rules and boundaries that govern system design

WHY IS ARCHITECTURE IMPORTANT?

Six Reasons Architecture Matters

Good software architecture directly impacts system quality:

- Maintainability — easier to update and extend code
- Scalability — handles growing users and data loads
- Security — enforces boundaries and access control
- Performance — optimizes data flow and processing
- Availability — supports fault tolerance and uptime
- Clear Organization — aligns teams and responsibilities

KEY PATTERNS OVERVIEW

Three Core Architectural Patterns: A Structured Introduction

Layered Architecture

Organizes the system into stacked horizontal layers, each with a specific responsibility. Each layer only communicates with the layer directly below it, ensuring separation of concerns and maintainability.

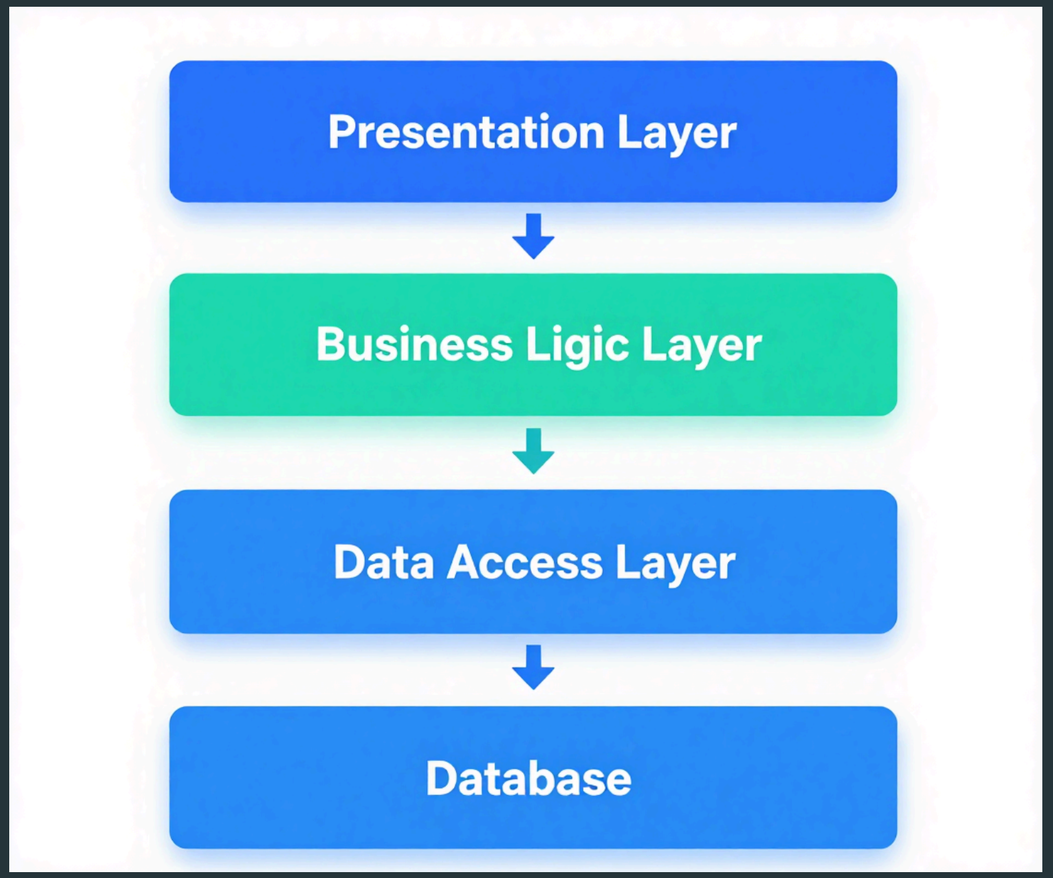
Client-Server Architecture

Divides the system into two roles: clients that request services and servers that process and respond. Communication occurs over a network using protocols such as HTTP, enabling distributed access.

MVC Pattern

Separates an application into three components: Model (data), View (interface), and Controller (logic). This division promotes modularity and is widely used in web application frameworks.

LAYERED ARCHITECTURE



Pattern 1: Layered Architecture Structure

Presentation Layer: handles UI and user interaction

Business Logic Layer: processes rules and workflows

Data Access Layer: manages queries and data mapping

Database Layer: stores and retrieves persistent data

Applications: banking systems, ERP, CRM, e-commerce, hospital systems, university portals, inventory management, and enterprise dashboards

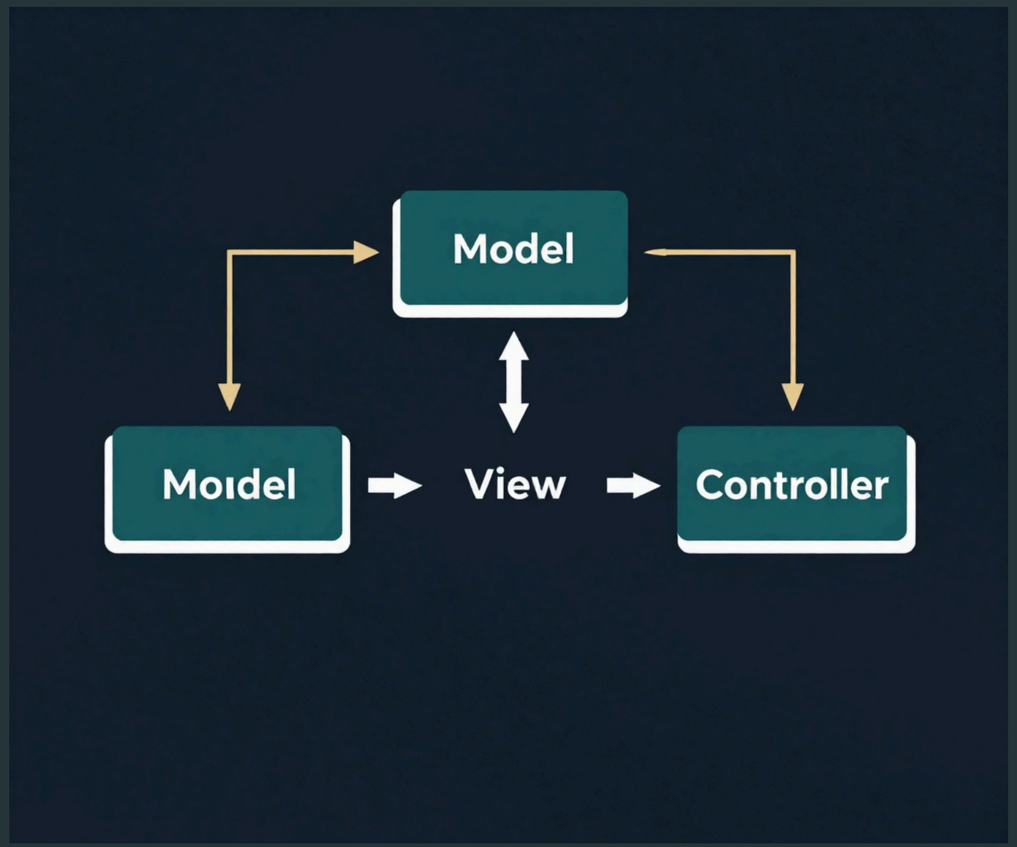
CLIENT-SERVER

Pattern 2: Client-Server Architecture

Client sends a request via HTTP/Network to the Server
Server processes the request and queries the Database
Database returns data; Server sends response to Client
Clear separation: client handles UI, server handles logic

Applications: web sites, online banking, email, cloud storage, streaming platforms, online gaming, mobile apps, and IoT systems

MVC PATTERN



Pattern 4: Model – View – Controller (Web Application)

Model: manages data and business rules
View: displays information and user interface
Controller: receives input and coordinates Model and View
Flow: User → Controller → Model → View → User

Applications: e-commerce websites, social networks, content management systems, learning platforms, booking systems, and administrative web portals

QUICK COMPARISON

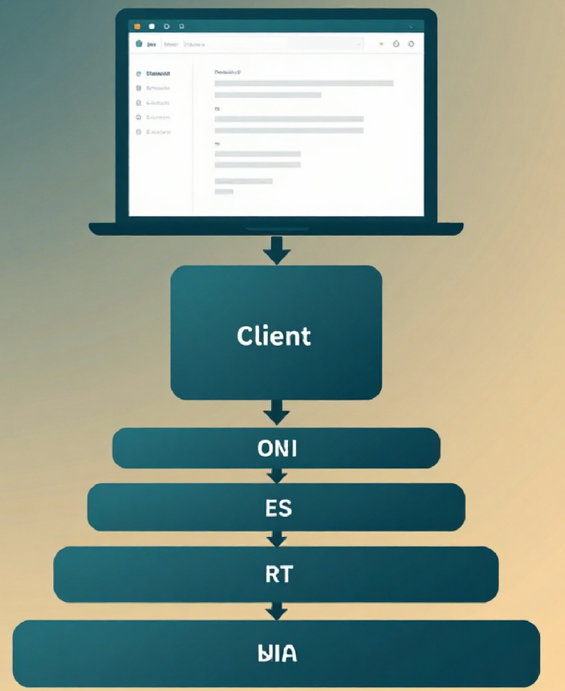
Layered vs Client-Server vs MVC — Side-by-Side Overview

LAYERED ARCHITECTURE
Main Idea:
Separate concerns into stacked horizontal layers
Structure:
Presentation → Business Logic → Data Access → Database
Coupling:
Each layer depends only on the layer below it
Typical Use:
Enterprise apps, banking systems, ERP platforms
Strength:
High maintainability and clear separation of concerns

CLIENT-SERVER
Main Idea:
Split system into service requesters and service providers
Structure:
Client → Network/HTTP → Server → Database
Coupling:
Loose coupling via network protocols
Typical Use:
Web applications, email systems, online services
Strength:
Scalability and centralized resource management

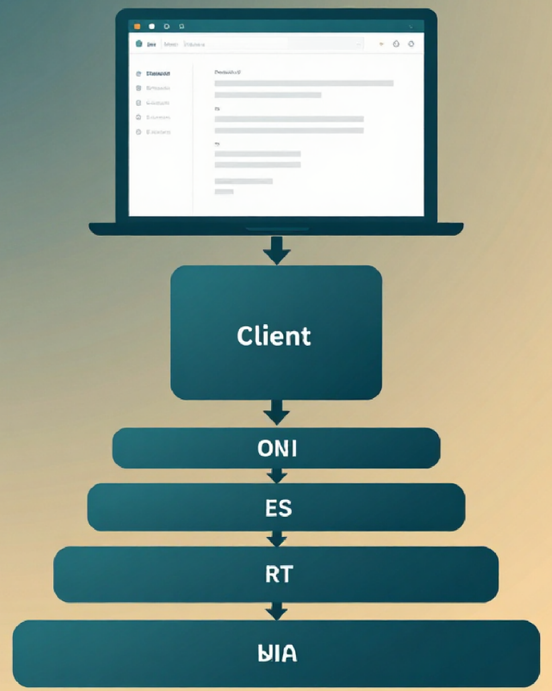
MVC PATTERN
Main Idea:
Separate data, UI, and control logic into three roles
Structure:
Model ↔ Controller ↔ View
Coupling:
Controller mediates between Model and View
Typical Use:
Web frameworks, desktop GUIs, front-end apps
Strength:
Parallel development and testable components

PRACTICAL EXAMPLE



- Layered Architecture: banking, ERP, CRM, e-commerce, hospitales, universidades, inventarios y dashboards empresariales.
-
- Client-Server: sitios web, banca en línea, correo, almacenamiento cloud, streaming, gaming, aplicaciones móviles e IoT.
-
- MVC: e-commerce, redes sociales, CMS, plataformas educativas, reservas y portales administrativos.

PRACTICAL EXAMPLE



Layered Architecture: banking systems, enterprise resource planning, customer relationship management, e-commerce platforms, healthcare facilities, educational institutions, inventory management, and business intelligence dashboards.

Client-Server: websites, online banking, email, cloud storage, streaming services, gaming platforms, mobile applications, and Internet of Things (IoT).

MVC: e-commerce, social networks, content management systems, educational platforms, reservation systems, and administrative portals.

CONCLUSION

- Software architecture provides the structure of a system, defining its components, responsibilities, communication, and data flow. A good architecture makes software easier to develop, maintain, and scale.
- Architectural patterns provide proven ways to organize software. Layered, Client-Server, and MVC architectures help separate responsibilities and make applications more organized, flexible, and easier to understand.
- There is no single best architecture for every project. The appropriate pattern depends on the system's requirements, size, complexity, security, scalability, and resources. The key is to choose the pattern that best fits the project's needs.

REFERENCIAS

- Bass, L., Clements, P., & Kazman, R. (2021). Software architecture in practice (4th ed.). Addison-Wesley Professional.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley Professional.
- Microsoft. (2026). Architecture styles. Microsoft Learn.
- Microsoft. (2026). Microservices architecture style. Microsoft Learn.
- Amazon Web Services. (2023, July 31). Implementing microservices on AWS. AWS Whitepapers.